

Developing Video Games with Elementary Adaptive Artificial Intelligence in Unity

An Intelligent Systems Approach

Apostolos Meliones and Ioannis Plas
Department of Digital Systems
University of Piraeus, Greece
meliones@unipi.gr, ioannisps59@gmail.com

Abstract—Video games have increasingly demonstrated a great deal of audiovisual realism, in par with the massive performance improvement of computer systems. At the same time, their Artificial Intelligence (AI) component falls short in terms of realism because it is usually based on non-adaptive methods. Adaptive AI mechanisms can help increase video game realism allowing the game to adapt in real-time to the game progress and the user behavior. Following a short overview of the progress of AI in video games in the past years, this paper highlights the creation of modern video games with basic and elementary adaptive game AI using the Unity game development framework. Particular emphasis is on the details of the AI component. First, a shooter game with basic AI is created. Finally, an action-adventure video game is created featuring elementary case-based adaptive AI. The objective in this game is to create enemies which are able to perceive changes in the environment and adapt their strategies accordingly. Proposed AI practices can migrate into relevant real world applications, such as video surveillance and intrusion detection systems, mission critical autonomous networked patrolling and/or save and rescue robots, vision and hearing assistive applications, intelligent video and behavioral analytics to detect and predict threats etc.

Keywords—game AI; adaptive game AI; case-based adaptive game AI; intrusion detection; networked patrolling robots; Unity

I. INTRODUCTION

For quite many years the processing power of a computer system was primarily exploited for the creation of better game graphics, until the debut of 3D graphics cards in the early 90s which freed the CPU from supporting computationally intensive graphics tasks and released CPU power to better support other game functions. One such function is Artificial Intelligence. Over the last two decades video games have increasingly demonstrated a great deal of audiovisual realism, in par with the massive performance improvement of computer systems. At the same time, their Artificial Intelligence (AI) component falls short in terms of realism because it is usually based on non-adaptive methods, which create several drawbacks. A major drawback of non-adaptive AI is that nothing can stop the player from finding the opponent weaknesses in the game. When these weaknesses are understood the opponent (computer player) becomes vulnerable ipso facto leading to the human player prevalence. Sooner or later the user will be able to defeat the opponent using a specific repetitive game strategy.

Adaptive AI mechanisms can help overcome the aforementioned drawback, i.e. allowing the video game to adapt to the game progress and the user behavior [1]. This way the game can “learn” the user moves and react accordingly, re-planning an optimal behavior. However, adaptive AI is rarely applied in practice, as it requires numerous trials to achieve the above goal. Moreover, several game developer teams worry that the application of adaptive AI can lead to uncontrollable and unforeseen behavior which can cause significant problems during game-play.

Nowadays, the commercial video games industry has evolved to a multi-billion-dollar market. Unlike in the 1980s and the early era of the game industry when only a couple of people could create the famous game titles of that era [2], modern games are developed by large multidisciplinary game project teams funded by large budgets, confirming remarkably the early prediction about the future state of game industry attempted in [2]. Video games can be categorized in several types based on their characteristics. Such a categorization is not always dogmatic because of the lack of commonly agreed categorization criteria. However, currently the categorization of video games as action, fighting, platform, shooter, adventure, action-adventure, survival horror, role playing (single player and massive multiplayer online RPGs), simulation and strategy games is vastly undisputed. Fighting, platform and shooter games are considered sub-categories of action games. Simulation games are further sub-categorized into the so called God games and vehicle simulations. Strategy Games are further sub-categorized into puzzle games, turn-based strategy games, and real time strategy (RTS) games.

Exemplary representative games in these categories are Street Fighter, Tekken and Mortal Kombat (fighting games), Super Mario Bros and Crash Bandicoot (platform games), franchise of Wolfenstein, Call of Duty and Bioshock (shooter games), franchise of Broken Sword, Gabriel Knight, Monkey Island and Heavy Rain (adventure games), franchise of Metal Gear Solid, Tomb Raider, Uncharted and The legend of Zelda (action-adventure games), franchise of Resident Evil and Silent Hill (survival horror games), franchise of Final Fantasy and Pokemon (single player RPGs) and World of Warcraft and Star Wars: The Old Republic (MMORPGs), Sim City (God games), Flight Simulator (vehicle simulations), Lemmings (puzzle games), Civilization (turn-based strategy games), Command & Conquer (RTS games), etc.

This paper is organized as: First, it presents an overview of the progress of AI in video games in the past years. Next, it highlights the creation of a photorealistic 3D shooter video game with basic AI using the Unity cross-platform game development engine [3]. Particular emphasis will be on the details of the basic AI component. Finally, an additional action-adventure video game will be created featuring adaptive state-of-the-art AI. The objective in this game is to create enemies which are able to perceive changes in the environment and adapt their strategies accordingly.

II. ELEMENTARY ADAPTIVE ARTIFICIAL INTELLIGENCE IN VIDEO GAMES

The term artificial intelligence, first coined by J. Mc Carthy et al. [4], refers to a sub-division of computer science addressing the design and implementation of computing systems that mimic elements of human behavior. Such systems demonstrate features such as elementary intelligence, adaptation, deduction, reasoning, context awareness, learning, capability of problem solving etc. Symbolic AI was the dominant paradigm of AI research until the early 90s, a collective term for AI research assuming that intelligence can be achieved by the manipulation of symbols. Symbolic AI is largely used for knowledge level programming (e.g. expert systems). Subsymbolic AI is aiming at reproducing human intelligence primarily exploiting methods and tools of applied mathematics. Such methods are able to synthesize intelligent behaviors through the successive self-organization of simpler structural components (e.g. a neural network for classification and clustering). In this decade Machine Learning methods are largely statistical methods.

The artificial intelligence in video games (henceforth referred to as Game AI) comprises the decision algorithms that define the behavior of game characters during game-play [5]. Early video games such as Pong and Pac-Man involved an elementary AI. Nowadays, several games (e.g. Batman: Arkham Knight, Metal Gear Solid V: The Phantom Pain) demonstrate complex detailed environments requiring a high performance from Game AI in order to accomplish the necessary tasks as well as challenge the user interest.

A. Game AI Objectives

According to [6] Game AI aims at achieving seven goals in order to deliver an entertaining game for the user:

- No obvious cheating: Game AI has to be effective without cheating.
- Unpredictable behavior: Game AI behavior should not be predictable by the user.
- No obvious inferior behavior: Game AI should not exhibit obvious inferior behavior against the user, so that it cannot be defeated easily by the user.
- Use of the environment: Game AI has to exploit in a smart way the characteristics of the game environment.
- Self-correction: Game AI should be capable of correcting its behavior in order to avoid repeating mistakes.

- Creativity: Game AI should be capable of generating novel solutions to unforeseen game circumstances.
- Human-like behavior: The behavior exhibited by game AI should be equivalent in complexity to the human behavior.

Out of these goals, only the first four ones are usually implemented to some extent by game developers. The rest of the goals are major topics for academic research. Another goal of game AI is to control the game characters in a way that their exhibited behavior is consistent and realistic within the game. This is not always similar to demonstrating a competitive behavior. For example, in a first-person shooter game it would not be realistic if the characters controlled by game AI were shooting target with 100% success. Game AI in such games is usually designed to make deliberate mistakes in order to demonstrate a more realistic behavior. Several modern complex games attempt to build an optimal game AI in order to achieve a consistent game environment [7]. As soon as an optimal or near-optimal game AI is achieved, difficulty scalability techniques are applied in order to downgrade the strength of game AI to provide the right challenge to the user.

B. Academic Approach in Game AI

Game AI as perceived by academic researchers refers collectively to techniques aiming at controlling the intelligent behavior of game characters. Typically, game AI comprises the complex behavior of game characters, such as long-term planning, predicting the opponent moves, etc. A typical research field involves machine learning and its application and validation within an adaptive game AI. The focus is on Spronck's 5th and 6th game AI objectives, i.e. self-correction and creativity. An adaptive game AI is capable of adapting to alternating situations and behavioral learning while the game is played. However, the integration of learning capability in video games is a particularly difficult task. Current machine learning techniques seem not ideal for game environments. They require numerous learning trials and extensive training in case specific knowledge of the evolving game environment is not available a priori, are prone to errors before adopting a successful behavior, cannot adapt fast enough, and are computationally intensive. Therefore, researchers quite often opt to design special mechanisms for the adaptation of game AI. Furthermore, game developers are skeptical about the reliability of adaptive game AI. The case of a human player who is deliberately exhibiting wrong behavior so that a weak game AI is trained is characteristic. Evidently, it is critical to focus on reliability of the applied techniques, which have to function according to strictly defined reliability requirements.

C. Non-Adaptive Game AI

The goal of game AI is to create the impression to human players that the opponent characters are controlled by other humans [8]. This is usually achieved by the use of well tested functional game AI techniques employed in modern video games, such as decision trees, finite state machines and scripting. Decision trees are usually implemented as a sequence of if-then statements that classify the character moves. Usually, the derived classification includes the decisions that have to be made. Decision trees are computationally efficient structures

which are easily applied and understood. Finite state machines (FSM) are behavioral models consisting of finite number of behavior states, transition between states and transition conditions. FSMs in today's games can be very large and consequently difficult to maintain. Alternatively, sequences of expected behaviors concerning given game states can be implemented with scripts which are usually written in a comprehensive scripting language (e.g. Lua). However, the larger the game complexity, the larger is the set of scripts of game AI. Therefore, quite often such codes suffer from design and programming mistakes [9].

The use of the above mentioned non-adaptive methods are sufficient for implementing game AI for simple or even complex games. However, they may not be adequate to cope with complex video game challenges requiring a game AI which should be consistent with an evolving complex game environment (e.g. in real-time strategy games). Nevertheless, Game AI research receives little or no attention by game developers, because a sophisticated AI component is not often a key factor for commercial success. Another reason is that it is difficult to setup a test bed to perform AI trials and train game AI in a complex video game [10]. Evidently, video game complexity will increase in the near future, requiring an AI component which will be able to adapt dynamically to rapidly varying conditions [11].

D. Adaptive Game AI

Today's video games demonstrate a complex realistic environment in which the characters controlled by the game AI should exhibit a human-like behavior. A characteristic of human-like behavior is its ability to adapt to varying circumstances in the game environment. This ability is termed *Adaptive Game AI*. When applied successfully, adaptive game AI is capable of correcting faults in the programmed game AI as well as search for proper anti-tactics against the actions of the human player. Recently machine learning techniques were successfully exploited to create an adaptive AI that can be applied in complex video games [11].

Different approaches were proposed for offline and online learning. Offline learning takes place while the game is not played by a human player and relies on observations regarding the playing tactics of a supposed human player. Online learning takes place while the game is played by a human player and relies on observations of human behavior. Online learning can be applied in order to adapt the game AI according to the experience level of the human player, adopting a similar tactic or strategy. For the time being only a few games integrate an online learning capability. This is because game developers are biased against an adaptive game AI which could have a negative impact on game behavior. It is though expected that online learning will be a typical feature of video games in the near future [11].

E. Case-Based Adaptive Game AI

Case-Based Reasoning (CBR) is a methodology for problem interpretation and solving, which is based on the explicit store and reuse of experiences and their generalization [12]. CBR problem solving relies on the real world domain observation that similar solutions apply to similar problems.

The CBR cycle comprises 4 phases: Retrieve, Reuse, Revise and Retain. In the retrieve phase, the system retrieves a subset of cases from the base scenario relevant to the current problem. During the reuse phase, the solving of retrieved cases is adapted into the problem framework. Next, the revise phase validates the proposed solution through real world trials or evaluation by experts. This phase provides feedback to the system regarding the correctness of the anticipated solution. Last, in the retain phase the system decides whether the resolved case should be added in the case base library.

Case-Based Reasoning can help implement a case-based adaptive game AI capable of creating character models and behaviors that react very efficiently in real gameplay conditions, based on past gameplay experiences. Environment knowledge is automatically collected by game AI and is exploited to create efficient behaviors during gameplay. Case-based adaptive game AI is expected to yield efficient adaptive game AI in today's complex online video games, as it will be able to retrieve and store numerous appropriate cases from many other gameplay experiences [13]. The research focus of case-based adaptive game AI is on the adaptation mechanism, the opponent modeling and the evaluation function (see Fig. 1). However, several restrictions have to be taken care: (1) Pre-installed in the game information is required to start the adaptation process. (2) Game AI cannot understand generic game environment information generated during gameplay. Instead, the game developer has to predefine specific information which game AI can understand and process during gameplay to adapt its strategy in real-time. (3) Game AI competitiveness against the human player should be moderated so that the game does not turn immediately at a competitive level in which it cannot be defeated by a human player. The case-based adaptive game AI technique has been applied already in the RTS video game titled "Spring", which demonstrates an effective adaptive game AI [14].

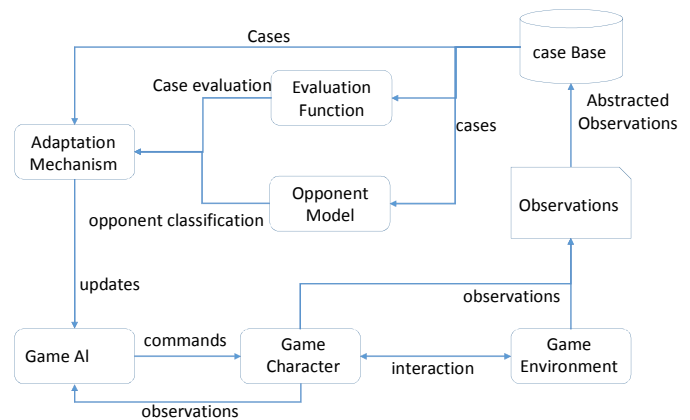


Fig. 1. Case-Based Adaptive Game AI.

III. GAME WITH BASIC ARTIFICIAL INTELLIGENCE

We next study two games we have developed using the Unity game engine. The first game is called "Danny's Nightmare" and integrates a basic AI component with a typical simple FSM illustrated in Fig. 2. The emphasis of the discussion will be on the details of the game AI component, being a paradigm of basic game AI practice.

Danny's Nightmare Finite State Machine

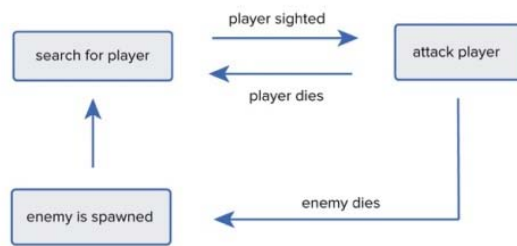


Fig. 2. Danny's Nightmare Game AI FSM.

A. "Danny's Nightmare" Gameplay

Gameplay is as follows: Danny is having a nightmare where his toys have come to life and are after him in order to devour him. His only hope is a machine gun which he must use for his survival against the creatures chasing him. The enemies are coming 'in waves' and the difficulty of defeating them increases gradually. In total the game consists of 10 waves with 3 different kinds of opponents. As the game develops, several power ups for Danny's machine gun appear, which make the extermination of the enemies easier. The space where the game evolves and game models are placed is depicted in Figs. 3-4.

B. 3D Models

Danny is a kid who wakes up in his nightmare in order to confront the monstrous toys that are after him. His fbx model is pretty representative: it presents a kid with pajamas and a cap.

ZomBear is the first enemy that Danny encounters in the game. This enemy is a teddy bear in huge form. The player can beat ZomBear easily, but it is very fast and sometimes can become a small threat.

ZomBunny is the second enemy Danny engages and it appears in the later waves. It is more difficult to defeat in comparison with ZomBear and faster than it.

Zelegant is the last opponent of Danny in the video game, kind as like 'the boss'. It is even harder to be killed compared to ZomBear and ZomBunny and it is capable of causing a great amount of damage to Danny.

C. Danny's Code

Danny's Movement: This section gives Danny's control to the player and adjusts his locomotion and the animations needed. It first sets the standard speed of the character with a float variable. It then stores in a vector the coordinates of the character in the area, so as to use them as reference later. Subsequently, it defines the distance of the camera from the player. Finally, it sets the functions related to forward and backward move, as well as the turning in the area. This last part is of high importance as it constitutes the overall functioning of the character controlled by the player.

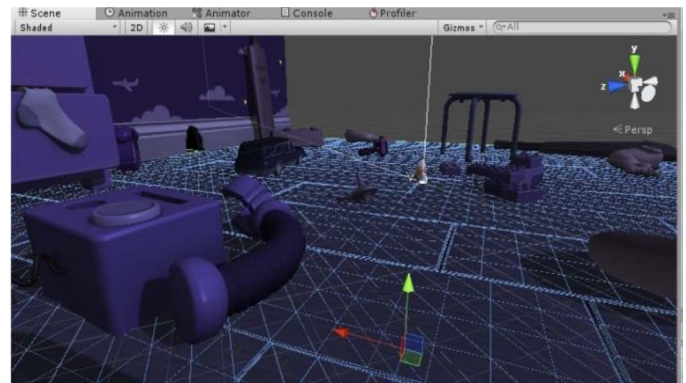


Fig. 3. Game 3D models placed at level's space.

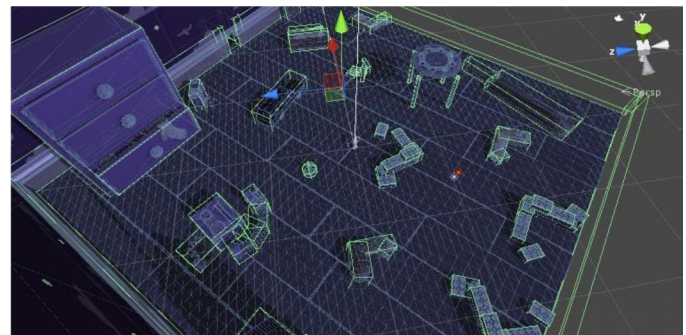


Fig. 4. Level's NavMesh.

Danny's Health: This section gives health to Danny. It sets prices for the initial health, the health at a particular moment, the time in which Danny does not take damage (this happens for a while after the character takes a hit) and the audible sounds after every hit or player's game over. Also, the health diminution functions are being set along with the damage and the gameover's animation.

Shooting: It enables Danny to shoot and defines the damage rate of the enemy, caused by the bullet. In addition, it determines the ammunition power ups as well as the extra damage they cause comparatively to plain bullets. Firstly, it designates the time in which the power ups will be active and the time where they will wear off and the ammunition turn normal again. Next, it sets the color of each power up and the animations and the effects displayed when the respective bullets are acting. Finally, it determines the bullet flow in a straight line against the enemies (raycast) and the moment that this effect wears off when the player stops shooting.

D. Managers

Game operation is executed via the following managers:

Gameover Manager: This function defines when the game will end after Danny's death. Level loading times and time after game over (which is needed in order to restart the game) are defined. It comprises a controller which checks if Danny's life bar drains empty. If a situation like this happens, the game state will be set to game over.

Pick Up Manager: It declares which power ups will appear to strengthen the player after a specific game score. After every 1500 points a power up will appear. These power ups can increase Danny's health bar or they can boost the damage power of Danny's rifle.

Score Manager: It calculates the game score. Through the awake function, score turns to zero and the function operates every time the game starts. Furthermore, the Update function defines how the score will be displayed at the screen and how it will be calculated.

Wave Manager: The Wave Manager defines the time distance in which an enemy wave starts to attack the player. The wave number increases as per the difficulty of the game. As a result, stronger opponents are spawned. A Spawn method is responsible for the spawn operation. In this method the NavMesh is being checked for an empty spot, and when this spot is discovered a new enemy is spawned. The only condition is that the spot in which the enemy will spawn must be not visible from the player. Ten waves appear in the game in total, each increasing the game's difficulty. The existence of enemies of the current wave is continuously checked. If even one enemy of the current wave is still alive, the wave number does not increase, otherwise the next wave appears.

E. Enemy's AI

In this specific game the enemy's AI is basic and does not include advanced functions. Game AI of a typical video game aims at simply chasing the player and damaging him. It does neither involve a decision system for re-evaluation of strategy nor for choosing the best attacking time (see Fig. 2).

Enemy Attack: This section deals with the attack on the player (see Fig. 5). It matches the first two characters (ZomBear and ZomBunny) and does not affect Zelephant's intelligence. First, it sets the time per attack. Next, it sets the damage made by each attack and 'tells' the enemy that the object to be attacked is the gameobject player. A check is executed to find if the player is in range (if it is within the field of vision) and if it is, the enemy hunts the player to damage him. It comprises 5 functions:

- *Awake* defines the references relative with the use of objects from other code sections in the operating script.
- *onTriggerEnter* detects the gameobjects in order to check if the gameobject is the Player. Then, the boolean variable *playerInRange* becomes true and the enemy perceives that it can lash out. The reaction time is set to 0.2f.
- *onTriggerExit*, where similarly the enemy detects the gameobjects once more, and in case it does not find the player then *playerInRange* becomes false and the foe stops chasing the player.
- *Update* calculates the duration of the assault and checks if the player is alive.
- *Attack*, which damages the enemy.

Enemy Health: This section defines the enemy's health, the score after the enemy's death, as well as the settings for the

appearing power ups. In the beginning, it defines the standard enemy health, a boolean variable representing if it is dead or not, and several variables related to sound, animation etc. It comprises 4 functions:

- *Awake* creates references which aim to transfer objects from other code section of the game in the specific script. Through this operation anything that had been previously created can be used again without re-creating it. In a nutshell, Awake repeats this particular functionality in several code sections.
- *Start* aims at initializing the health of the foes.
- *Update* refreshes the Burning Animation of the enemy.
- Finally, *TakeDamage* deducts the enemy's health, and if it is zero or subzero the enemy is dead. Additionally, at this point it incorporates the power ups which randomly spawn after the defeat of a foe. As time goes by, the chance of a power up appearing increases. The probability of a Health Pickup, Bouncing Bullets or Piercing Bullets power up appearing is 50%, 30% and 20% respectively.



Fig. 5. Danny is fighting the hordes of enemies.

Enemy Movement: This section deals with the movement of the two enemies. It defines a layermask for the raycast, so as to hit only objects at the shootable layer and designates the *roamSpeed* and *attackSpeed* of the foes. Following the creation of References for the player's place in the current level, his health and the foes health, it materializes three functions:

- *Awake* detects the gameobjects and sets references related to the player's place in the level, his health, the foes health and the navigation mesh. Remember that without the references we would not be able to use other code functions at this point.
- *Update* checks if the enemy is alive and then sets its speed. It also calculates the distance between the enemy and the player.
- *SetRandomNavTarget* perceives the location of the enemies in the space and makes them move randomly when they have not detected Danny.

Zelephant Attack: Zelephant's code is similar to the other enemies, with the exception of the function *SpecialAttack*. Via *SpecialAttack*, a Zelephant jumps and when it touches the ground it unleashes bullets everywhere around it and continues moving within space. This function checks the Zelephant place. If it is on the ground, the time of next jump is calculated.

If it is on air, it unleashes the bullets after its landing. It first sets the time in between successive attacks, the number of bullets in each attack and the angle between bullets. Next, the Zelephant perceives the player's presence through the GameobjectPlayer reference, as well as if the player is in range.

- *Awaken* aims at searching the player in order to find him and begin the foe's move.
- Subsequently, *Update* refreshes the timer and checks if it is time for the Zelephant attack.
- Finally, it activates the sounds the enemy makes during an attack and the *Special Attack* is being designated, as described before.

IV. GAME WITH ADAPTIVE ARTIFICIAL INTELLIGENCE

The second game is called "Escape from Lab#98" and integrates an elementary case-based adaptive AI component, being a paradigm of more sophisticated AI which can be useful as well in similar scope real world intelligent applications.

A. "Escape from Lab#98" Gameplay

Tony is in custody in a lab secured by robots. Tony must escape from the laboratory, but in order to do so he has to hide from the robots and the cameras guarding and watching every corner. The only road of hope is an elevator which leads outside, but Tony has to find an access cart to access it. As if these were not enough, lasers block Tony's path. These lasers deactivate from some panels which must be hacked.

B. 3D Models

Tony is the main character of the video game, controlled by the player. He is a child in the future, as his clothes reveal, and the fbx model is designed accordingly.

Robots are the only foes of Tony in the game and are controlled by game AI. They carry a weapon that is capable of knocking Tony unconscious.



Fig. 6. Floor plan of the game.

C. Game Environment

Following the placement of objects that create the suitable surroundings for Tony's movement and coverage, the room is baked and NavMesh is created. Through NavMesh the characters can be programmed to perceive where to move and where not. Without NavMesh the characters would pass through walls, would walk above the ground and would be off limits. This particular stage is significantly more complex than the level of the previous game because of the number of the functions and demands. The demands are related with the creation of cover spots for Tony, as well as the hallways which have a mazy element to raise the difficulty. The paths have

various obstacles; cameras and laser fences. Tony can switch off the Laser fences through the panels but he cannot do something for the cameras. If they see him, all the foes are notified and coming at him. The only thing Tony can do is to not be seen by the cameras, passing when they are looking opposite. Fig. 6 illustrates the created space with the starting point and the finishing point.

Panels are on top of terminals (Fig. 7). They are machines which must be hacked by the player or unlocked by entering the right code in order to be deactivated. Such codes constitute of dispersed numbers in the game stage and the player has to find them so that the game can proceed. This process is the only puzzle in the game.



Fig. 7. Last terminal before the elevator is reached.

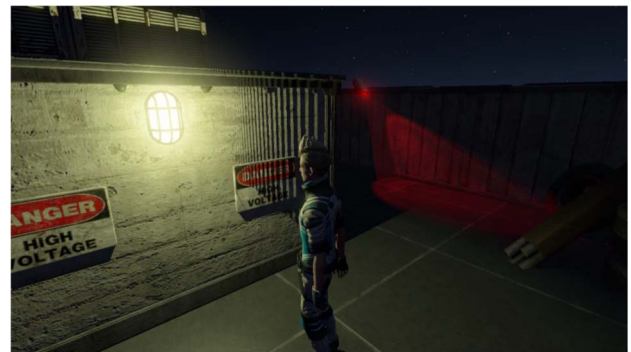


Fig. 8. Tony remains unseen by the camera.

Cameras and laser fences are barriers and are located in key spots relevant to the player's progress. They are usually found in corners or blocking the access in locations of high interest in the map (Figs. 8 and 9).



Fig. 9. Tony in front of a laser fence.

Finally, the elevator is where Tony escapes and the game ends (Fig. 10). But for the elevator to operate, the player must collect a card hidden in the laboratory.

D. Enemy AI

In this section, a simple version of Case-based Game AI has been specified consisting of If rules. First, several variables are created, related with the patrolling speed of the robots, the speed for chasing Tony, the estimated waiting time until the next persecution etc. Afterwards, a structure of references is created. This structure affects the next scripts concerning the vision, player's lifespan, pursue time and the last point where the player was perceived by the robots. The code includes 4 functions; Awake, Update, Shooting, Patrolling.



Fig. 10. Tony reached the elevator escape point. A laser fence can be seen in the distance.

- *Awake* defines the references related to the enemy's vision and their movement. It also matches the *PlayerHealth* with the *GameObject* of the player and defines a variable entitled *LastPlayerSighting* which indicates the position where the player was last seen in the space.
- *Update* detects if the player is alive. If not, the robots continue the patrolling. If yes, the game checks if the player is in range for the foes to shoot him. If the player has activated the alarm, the variable *Chasing* becomes true and the enemies start to investigate the area where the player had triggered the alarm. In the end of Update, if Tony makes a sound or he is sighted by the robots, they start the hunt. If none of this happens, the area robots continue the patrolling.

In this part of the code a Case-based Game AI is applied, which receives certain information from the environment and uses it to recognize the player's location. The code includes many If rules, which make the Game AI decide next moves.

- *Chasing* includes the chasing code. The moment the player is aware by the robots or the cameras, a vector is created from the enemy's position until the place where Tony was spotted. Foes are searching for Tony in that area (Fig. 11).
- In *Shooting*, the robot freezes and shoots Tony. This specific mode takes place only if it is activated by Tony's view (Fig. 12).

- Last is the *Patrolling* function. If the robot is close to a waypoint or if there is nothing to search, the patrolling time increases. If the patrolling time exceeds the waiting time, then the robot's waypoint changes so as new patrolling paths are created.



Fig. 11. A robot is chasing Tony.

Enemy Sight: Robots are able to see and hear, thus a realistic AI is created. In the ensuing, the Enemy Sight of Game AI is analyzed.

First, a view field for the robots is set, as well as a shooting range. Three variables are defined, which consider if the player is visible or hearable and whether he is within shooting range. Subsequently, some references are created related with the perception of Navigation Mesh by the robots, the collider's and player's presence in space, as well as the interconnection of the player with this part of the code. Furthermore, the following functions are materialized;

- *Awake* defines the references activated by Navigation Mesh, Collider and Animator in order for the enemy to move and be perceptible by the rest entities of the stage.
- *Update* checks whether the player is alive and visible to the enemy.
- *OnTriggerStay* checks if the player has activated a trigger. In that case a vector is created from the enemy up to the player. If this vector is smaller than the half of the vision vector and if the raycast sprayed touches the player, then the player is visible from the foe. Automatically, the last spot where Tony was seen is stored as the current position.
- *OnTriggerExit* detects if the player is away from trigger zone. In that case he is no longer visible.
- *CalculatePathLength* creates a path based on Tony's position. Later, a chart is created with spots which are the number of the corners in the path. The first spot is the enemy's place and the last is the player's. The spots in between these two are the corners of the path. After this, the path is stored in the variable *PathLength* and its length grows by a quantity equal to the distance between each waypoint and the next.

Enemy Shooting: The last part of the code deals with the way robots shoot against Tony.

Initially, the variables concerning the damage that the player takes are set, as well as the settings of the visual effects

of the bullet. Next, references are created related to the laser which aims the player, the colliders, the boolean variable Shooting representing the state that the robot is shooting that moment, as well as the player's place in the stage. The following functions are materialized.



Fig. 12. A robot is about to shoot Tony.

- *Awake* regulates the mentioned references and estimates the scaled damage, which is the difference between maximum and minimum damage.
- *Update* calculates the shooting procedure and *OnAnimatorIK* poses the robot's hand to aim Tony.
- The *Shoot* function includes a Boolean variable (*Shooting*) which becomes true, whereas the distance from Tony is calculated by the foe's distance from the sphere collider of the player. Finally, the damage is estimated, the player takes the damage and the shooting effects appear.
- The last function *ShootEffects* starts the line of the renderer from the gun's muzzle until the mass center of Tony. The line becomes true and the bullet animation is played from the muzzle towards Tony.

Escape from Lab #98 Finite State Machine

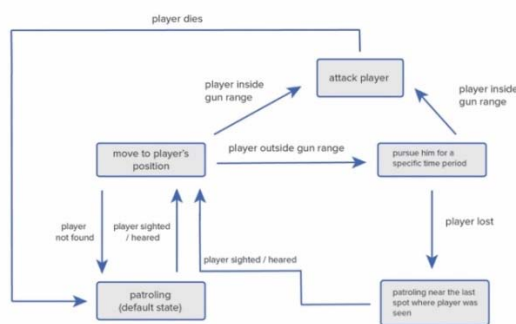


Fig. 13. Escape from Lab#98 Game AI FSM.

E. Basic AI vs Case-Based Adaptive AI

A simplified version of case-based adaptive AI is applied in "Escape from Lab#98", illustrated in Fig. 13. The AI component in this game has been designed to adapt to the game conditions and draw upon the environment information in order to redefine its strategy. Robots patrolling the lab space are following a specific route, unless they detect Tony and start

chasing him. In case they do not find Tony they change their patrolling route which is steered around the last point in which Tony was detected. The game AI is therefore able to redefine its strategy through adopting a method which hinders the user progress towards the completion of the game. Unlikely, the basic AI in "Danny's Nightmare" is simple and cannot redefine its strategy against the player. The enemies are capable of detecting the player and collectively attacking him. An enemy detecting the player is moving towards him with the objective to remove life points from Danny. The sole intelligent feature of enemies is their capability to bypass obstacles on their way to reach the player.

V. CONCLUSIONS

This paper highlighted the creation of modern video games with basic and elementary adaptive game AI using the Unity cross-platform game development engine. Particular emphasis was on the details of the AI component. First, a shooter game with basic AI was created. Finally, an action-adventure video game was created featuring elementary case-based adaptive AI, in which enemies are able to perceive changes in the environment and adapt their strategies accordingly. The developed AI code can be migrated into relevant real world applications, such as cognitive video surveillance and intrusion detection systems, mission critical autonomous networked patrolling and/or save and rescue robots, vision and hearing assistive applications, intelligent video, hearing and behavioral analytics to detect and predict threats, etc.

REFERENCES

- [1] S. Bakkes, P. Spronck, and J. van den Herik, "Rapid and Reliable Adaptation of Video Game AI" IEEE Transactions on Computational Intelligence and AI in Games, Vol. 1, No. 2, June 2009.
- [2] S. Collins, "Game Graphics during the 8-bit Computer Era", ACM SIGGRAPH Computer Graphics, Vol. 32, No. 2, 1998.
- [3] Unity Game Engine, <http://unity3d.com>
- [4] J. McCarthy, M. Minsky, N. Rochester, C. Shannon, A Proposal for the Dartmouth Summer Project on Artificial Intelligence, 1955.
- [5] A. Nareyek, Artificial Intelligence in Computer Games: State of the Art and Future Directions", ACM Queue, 1(10): 58-65, Feb. 2004.
- [6] P. Spronck, "Adaptive Game AI", PhD Thesis, Maastricht University Press, Maastricht, The Netherlands, 2005.
- [7] M. Buro, T. Furtak, "RTS Games as Test-Bed for Real-Time AI Research", ResearchGate, December 2008.
- [8] J. Laird, M. van Lent, "Human-level AI's Killer Application: Interactive Computer Games", AI Magazine, 22(2):15-25.
- [9] A. Nareyek, E. Freuder, R. Fourer, E. Giunchiglia, R. Goldman, H. Kautz, J. Rintanen, A. Tate, "Constraints and AI Planning", IEEE Intelligent Systems, March/April 2005, pp. 62-72.
- [10] M. Buro, "Call for AI Research in RTS Games", Proceedings of the AAAI-Workshop on Challenges in Game Artificial Intelligence, pp. 481-484, AAAI Press, Menlo Park, California, USA.
- [11] S. Tekofsky, P. Spronck, M. Goudbeek, A. Plaat, and J. van den Herik, "Past Our Prime: A Study of Age & Play Style Development in Battlefield 3", IEEE Transactions on Computational Intelligence and AI in Games, Vol. 7, No. 3, pp. 292-303, 2015.
- [12] I. Watson, F. Marir, "Case-Based Reasoning: A Review", The Knowledge Engineering Review, 9(4): 327-354, Dec. 1994.
- [13] P. Spronck, J. van den Herik, "Game Artificial Intelligence that Adapts to the Human Player", ERCIM News, No. 57, April 2004.
- [14] S. Bakkes, P. Spronck, J. van den Herik, "Opponent Modelling for Case-Based Adaptive Game AI", Entertainment Computing, 1(1): 27-37, January 2009.